

The C++ Firmware to Networking Syllabus (Phases 1-5)

Phase 1 (Completed): Fundamentals & Memory Mechanics

This phase established the procedural foundation of C++ and the critical shift from C-style raw pointers to C++ references.



The Cherno Video Block (Completed)

1. **Variables, Data Types, & Functions:** The absolute basics of C++ syntax.
2. **Header Files & Translation Units:** How the compiler and linker actually see your code.
3. **Conditions, Loops, & Control Flow:** Standard execution branching.
4. **Pointers:** Memory addresses and the realities of raw memory access.
5. **References:** The C++ alternative to pointers (pass-by-reference).



The Textbook Block (Completed)

- **A Tour of C++:** Chapter 1 (The Basics).



The Phase 1 Boss Fights (Completed):

- **Mini-Project 1: Procedural Ring Buffer.** (Mastering modulo arithmetic and C++ reference parameters without classes).
- **Mini-Project 2: Sensor Data Logger.** (Handling nested structs, pass-by-const-reference, and simulated asynchronous data flows).

Phase 2 (Part 1 - Completed): Object-Oriented Architecture

This block introduced the core concepts of Object-Oriented Programming (OOP) and how to design abstract hardware contracts using Polymorphism.



The Cherno Video Block (Completed)

1. **Classes vs Structs in C++:** The C++ default visibility rules.
2. **Inheritance in C++:** Building hierarchical class structures.
3. **Virtual Functions in C++:** Dynamic dispatch and the VTable memory cost.
4. **Interfaces (Pure Virtual Functions):** Creating strict hardware contracts (= 0).
5. **Visibility in C++:** Controlling memory access with `public` , `private` , and `protected` .



The Textbook Block (Completed)

- **A Tour of C++:** Chapter 2 (User-Defined Types) and Chapter 3 (Modularity).



YOU ARE HERE: Mid-Phase 2

You understand Pointers, References, Inheritance, Virtual Functions, Interfaces, and Visibility.

- **Your current power:** You can design a polymorphic software architecture (like the Sensor HAL).
- **Your current weakness:** You don't quite know the modern C++ rules for safely constructing, moving, and destroying those objects in memory.

Phase 2 (Part 2 - Completion): Object Mechanics

This phase is about learning the internal plumbing of a C++ `class`. These videos teach you how objects are spawned, how they behave, and how to protect their internal memory.

The Cherno Video Block (Watch these next)

1. **Member Initializer Lists in C++:** (Crucial. Shows how to initialize hardware without wasting clock cycles on garbage data).
2. **const in C++:** (How to create read-only functions. Firmware engineers use this everywhere).
3. **The Mutable Keyword in C++:** (A rare override for `const`. Good to know, rarely used).
4. **Ternary Operators in C++:** (Just syntax sugar for `if/else`).
5. **How to create/instantiate objects in C++:** (The difference between spawning an object on the Stack vs the Heap).
6. **The `new` Keyword in C++:** (C++'s version of `malloc`).
7. **The `this` keyword:** (How an object references its own memory address).
8. **The Arrow Operator in C++:** (Using `->` to access member functions via pointers).
9. **Implicit Conversion and the Explicit Keyword in C++:** (How to prevent C++ from accidentally converting an `int` into your custom object).
10. **Operators & Operator Overloading in C++:** (How to make `sensor1 + sensor2` actually compile).

The Textbook Block

- **A Tour of C++:** Read Chapter 4 (Classes). This formalizes everything you just watched.

The Phase 2 Boss Fights:

- **Mini-Project 3: Polymorphic Sensor HAL.** (You now have all the tools to build the `ISensor` project).
- **Mini-Project 4: Mock Register Bank.** (Mastering Operator Overloading, `const` methods, and the `this` pointer by building a safe, object-oriented wrapper around simulated memory-mapped hardware registers).

Phase 3: Modern Memory & The STL (The gRPC Bridge)

Do not skip this. If you jump to gRPC without Phase 3, you will drown. gRPC does not use raw C arrays or `malloc`. It uses the C++ Standard Template Library (STL) and automatic memory management.



The Cherno Video Block

1. **Arrays in C++:** (Standard C++ arrays).
2. **How strings work in C++ & String Literals:** (Crucial! Protobuf uses `std::string` for all text payloads. You must understand how they dynamically allocate heap memory).
3. **The `auto` Keyword in C++:** (Crucial for iterating through STL containers without typing massive type names).
4. **Dynamic Arrays in C++ (`std::vector`):** (Crucial! Protobuf uses `std::vector` for all array data. You must know how to `push_back` and `iterate`).
5. **Templates in C++:** (Required to build the Generic Ring Buffer. Bypasses writing the same class twice for `int` and `float`).
6. **Object lifetime in C++ (Stack/Scope Lifetimes):** (The foundation of RAI - Resource Acquisition Is Initialization).
7. **Smart Pointers in C++:** (Mandatory for gRPC. `std::unique_ptr` and `std::shared_ptr` automatically delete memory).
8. **Copying and Copy Constructors in C++:** (How to prevent accidental shallow copies of network buffers).
9. **Ivalues, rvalues, and Move Semantics:** (How to hand off a 10KB Protobuf message to a network socket without copying it).
10. **Casting in C++:** (Pay special attention to `reinterpret_cast` . This is how you convert structured objects into raw bytes for network transmission).
11. **Lambdas in C++:** (Inline anonymous functions. Heavily used in gRPC asynchronous callbacks).



The Textbook Block

- **A Tour of C++:** * Chapter 5 (Essential Operations: Copy/Move)
 - Chapter 6 (Templates)
 - Chapter 9 (Strings)
 - Chapter 11 (Containers: specifically `vector`)
 - Chapter 13 (Smart Pointers)



The Phase 3 Boss Fights:

- **Mini-Project 5: The Modern Generic Buffer.** (Refactoring your Ring Buffer into a Template Class utilizing `std::unique_ptr` and `std::vector`).
- **Mini-Project 6: Telemetry Dispatcher.** (Using `std::vector` , `std::unique_ptr` , and Lambdas to manage a dynamic registry of polymorphic sensors and filter their outputs—the ultimate prep for Protobuf serialization).
- *Note: This is the final project in your C++ Syntax learning repository.*



Phase 4: Ultra-Fast Local IPC (gRPC, Protobuf & Unix Sockets)



MILESTONE: REPOSITORY SPLIT *At this point, you will freeze your current C++ Workspace. You will create a brand new repository (e.g., `dock-comstack`) to act as the prototype for your actual*

production systems.

For your specific hardware (embedded Linux / Raspberry Pi), you will architect a stack combining **gRPC + Protobuf + Unix Domain Sockets (UDS)**.

Environment Constraints & RPi5 Deployment

- **Development:** You will be coding and compiling natively on WSL2 and your Linux Laptop.
- **WSL Danger Zone:** Unix Sockets (`.sock` files) do not safely cross the Windows/Linux file system boundary. When creating your socket in WSL, **always** place it in the Linux root file system (e.g., `unix:///tmp/telemetry.sock`). Never place it in `/mnt/c/...` .
- **Deployment (RPi5):** Because the RPi5 runs a Debian-based Linux OS, this exact setup is 100% portable. You will simply push your repo to the Pi, and run CMake directly on the Pi's ARM64 processor. The Unix Socket will behave identically.

The gRPC Unix Socket Workflow

1. **The Contract (`.proto`):** Define your data structures (Messages) and functions (Services).
2. **The Build System (`protoc`):** Configure CMake to run the Protobuf compiler.
3. **The Server:** Inherit from the auto-generated gRPC base class. Bind the ServerBuilder to `unix:///tmp/dock_comms.sock` .
4. **The Client:** Initialize a gRPC Channel pointing to that exact same `.sock` file.

The Final Boss Fight:

- **Mini-Project 7: The UNIX Distributed Comstack.** * Write a `telemetry.proto` defining a `DroneDock` service.
 - Build a Server executable hosting a gRPC server on `unix:///tmp/telemetry.sock` .
 - Build a Client executable that connects to the Unix socket and requests the data.

Phase 5: The Firmware Professional (Beyond Phase 4)

While Phase 4 focuses heavily on networking architecture, your pure C++ education will naturally evolve to solve the real-time constraints of robotics and hardware. Once the gRPC Comstack is functional, you will transition to learning:

1. **C++ Concurrency (Multithreading):**
 - *The Problem:* Your gRPC server blocks the main thread. How do you read hardware sensors simultaneously?
 - *The Curriculum:* `std::thread` , `std::mutex` , `std::lock_guard` , and `std::atomic` . You will learn how to run comms on Core 0 and sensor polling on Core 1 of the RPi5 safely.
2. **Unit Testing Frameworks:**
 - Transitioning from manual `printf` checks to enterprise CI/CD.
 - *The Curriculum:* Integrating **GoogleTest (GTest)** or **Catch2** into your CMake build to automatically verify your Ring Buffers and Comstacks.
3. **Modern C++ Standards (C++20/23):**

- When you are ready for maximum efficiency, you will explore C++20 features like **Concepts** (to replace messy templates) and **Coroutines** (for extreme high-performance asynchronous networking).